

RuleForge

Learning a New Skill from a Few Demonstrations

An interactive educational experiment on Skill Acquisition from Demonstrations and its connection to BDH-CQ.

The claim

A system can infer a previously unseen task rule from a small set of demonstrations and apply that rule to a new example without changing its model parameters at inference time.

Why RuleForge exists

Many explanations of few-shot learning stop at the phrase “the model learns from examples.” RuleForge makes the phrase testable. A learner sees input-output grids, proposes the hidden rule, and then receives an unseen input. The system shows the prediction beside the expected answer so the learner can see whether the inferred rule actually generalizes.

The selected concept is Skill Acquisition from Demonstrations, an approved topic in the DataForge Pathway brief. The brief asks for an explainer where the learner changes something meaningful, observes the concept behaving, and encounters a substantive BDH or BDH-CQ connection. [Brief]

1. The experiment

RuleForge starts with a small mystery rather than a definition. The learner sees several input-output pairs and has to explain what changed. In the example below, blue cells containing the value 1 become red cells containing the value 2. Their positions do not move.

Example rule: every 1 becomes 2; 0 cells stay 0.

COLOR **BEGINNER**

Color Transformation: Azure to Red
Examine how the visual token values transform between input and output.

Hint: Look closely at the individual cell colors. Does the position matter, or just the color value?

Observed Input-Output Demonstrations **3 DEMONSTRATIONS**

Demonstration 1

INPUT OUTPUT

Demonstration 2

INPUT OUTPUT

Demonstration 3

INPUT OUTPUT

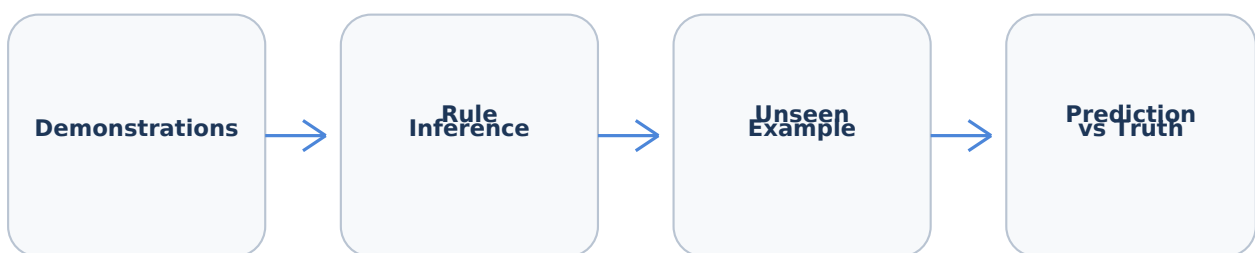
Step 1: Inspect & Hypothesize
Examine the demonstration pairs above. What operational rule turns each input into its output?

Check My Rule

Step 2: Systematic Rule Inference **Infer Skill**

Figure 1. RuleForge's demonstration-and-hypothesis interface. The learner first inspects the examples and writes the rule in their own words.

The learner then selects Infer Skill. RuleForge applies its transparent rule engine to the demonstrations and tests the inferred transformation on an unseen example. The important evidence is the predicted grid, the ground truth, and the cells that agree or disagree.



All transformations in the learning loop are computed from the selected examples.

Figure 2. The core learning loop. Changing the evidence changes the inferred transformation and the resulting prediction.

2. From a correct example to actual generalization

A rule is useful only if it works beyond the examples that revealed it. RuleForge separates demonstrations from the unseen test input. The learner cannot simply copy an output; they must apply the inferred transformation to a new grid.

The generalization section lets the learner vary the amount and content of evidence. The point is not a fixed headline percentage. It is the relationship between evidence and behavior: sparse examples can leave several rules plausible, while additional examples can remove ambiguity.

What counts as evidence? The project records the actual prediction produced by the rule engine and compares it with stored ground truth. If a chart is included in the deployed artifact, its values should come from those task results rather than typed-in numbers.

A useful failure case

Suppose one demonstration can be explained by both a color substitution and a position-dependent rule. One example is not enough to distinguish them. RuleForge treats this as a teaching opportunity: the learner can add or change a demonstration and see whether the competing explanations remain possible.

Too little evidence	More evidence
Several rules can fit the same demonstration.	A second or third example can rule out alternatives.
A plausible guess may still fail on the test input.	The unseen example reveals whether the rule generalizes.

This is deliberately small. The goal is not to reproduce the full difficulty of ARC-AGI. The goal is to create a fast, inspectable setting in which a learner can reproduce the central idea in under a minute.

3. How RuleForge is implemented

The first version uses a deterministic educational engine. That choice is deliberate. A learner should be able to trace why a prediction was produced instead of depending on an external language-model response that may change from run to run.

Frontend: React + TypeScript + Vite provides the interactive grids, demonstration cards, rule input, results, challenge, and reflection pages.

Backend: Python + FastAPI exposes the inference and evaluation functions.

Data: task definitions and ground-truth outputs are stored as JSON so the examples are easy to inspect and reproduce.

Rule engine: the initial task set uses explicit rule families such as color mapping, rotation, reflection, movement, counting, copying, deletion, and symmetry. The first release should keep the supported families small enough to explain.

A simple color-mapping rule can be represented as a mapping from an observed input value to an output value. The engine gathers these mappings across demonstrations, marks conflicting mappings as ambiguous, applies consistent mappings to the unseen input, and compares the result with ground truth.

```
Example logic:  
infer(demonstrations) → rule  
apply(rule, unseen_input) → prediction  
evaluate(prediction, ground_truth) → correctness + cell accuracy
```

This is a toy educational model. It is not an official BDH-CQ implementation and its results must not be presented as results of the BDH-CQ research model. The Pathway brief explicitly allows a toy or independent reimplementation when it is clearly labeled and not presented as official BDH. [Brief]

The implementation teaches the behavioral idea—acquiring a task rule from demonstrations—while the BDH-CQ section explains how the research system approaches the same broad problem using recurrent contextual state.

4. Why BDH-CQ belongs in the story

The connection to BDH-CQ is not an extra “AI” page added after the experiment. It answers the question raised by the experiment: if a system receives a new task through demonstrations, where does the task-specific information live?

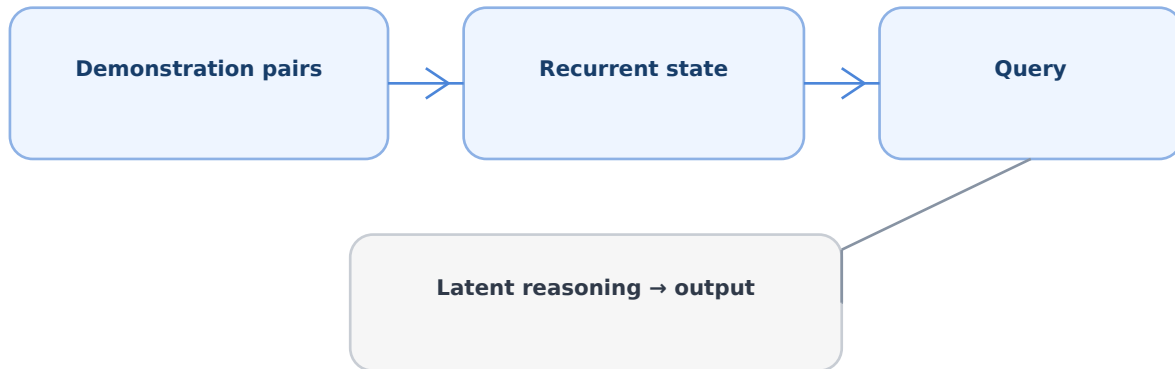


Figure 3. Conceptual BDH-CQ flow. Demonstrations update recurrent memory; the query is then processed using that task-conditioned state. This is a teaching abstraction, not a reproduction of the full model.

The 2026 BDH-CQ paper describes a model that combines in-context learning with recurrent latent reasoning. Inputs presented during inference update recurrent memory, after which the model solves the query through iterative computation in a high-dimensional latent space. The paper evaluates the approach on ARC-AGI-1 and uses controlled ARC-like interventions to study what the model learns from demonstrations. [1]

The important distinction for RuleForge is where adaptation happens. In the BDH-CQ setting described in the project brief, evaluation-task demonstrations are not used to update model parameters at inference; task information is accumulated in recurrent state. [Brief]

RuleForge does not claim to reproduce that architecture. It gives the learner a small observable analogy: demonstrations provide task evidence, a task-specific representation is formed, and that representation is used to produce a new answer. The research section makes clear that BDH-CQ performs this process with a learned recurrent architecture and latent computation, not with RuleForge's hand-designed mapping engine.

Evidence level: RuleForge behavior is directly computed by the project. BDH-CQ mechanism and evaluation claims are research claims sourced from the primary paper. The diagram is an educational simplification.

5. What the learner should leave with

After using RuleForge, a learner should be able to explain the difference between seeing examples and memorizing outputs. They should be able to describe a demonstration as evidence about a hidden task rule, test a rule on an unseen input, and explain why additional demonstrations can resolve ambiguity.

They should also be able to explain the BDH-CQ connection at the right level: the model uses inference-time context to update recurrent memory and then performs latent recurrent computation, while model parameters remain fixed during that adaptation. The learner should not conclude that RuleForge itself is BDH-CQ.

Limitations

RuleForge uses a deliberately constrained task family. Its deterministic rules are much simpler than transformations found in the full ARC-AGI benchmark. It does not establish that a general AI system can learn arbitrary skills from a few demonstrations. It is an educational substrate for making one claim observable and testable.

The project also separates two ideas that are easy to mix together: rule induction and reasoning execution. A system may infer the right rule but apply it incorrectly, or apply a simple rule perfectly while failing on a task whose hidden structure was not captured by its rule family. Showing this distinction is more useful than hiding failures behind a single score.

6. Reproducing the experiment

The public repository should contain the frontend, backend, task JSON files, inference engine, evaluator, research notes, setup instructions, source-and-license record, and AI disclosure. A new user should be able to install the dependencies, start the API, start the frontend, open a supplied task, and reproduce the same prediction from the same demonstrations.

The public artifact should open without sign-in. The interaction should begin with a working preset, respond quickly to changes, keep important state visible, and show the expected answer beside the model output. These are direct design expectations in the Pathway brief. [Brief]

The supporting package also includes a separate one-page concept summary. It should not be merged into this blog: the brief describes the concept summary as a self-contained technical briefing rather than a blog post. [Brief]

7. Primary sources

[1] Engdahl, B., Kosowski, A., Chorowski, J., Stamirowska, Z., Uznański, P., Jiang, J., Phadke, R., Kinas, R., & Zhong, R. (2026). BDH-CQ: In-Context Learning with Recurrent Latent Reasoning. arXiv:2608.09888.

<https://arxiv.org/abs/2608.09888>

[2] Kosowski, A., Uznański, P., Chorowski, J., Stamirowska, Z., & Bartoszkiewicz, M. (2025). The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507.

<https://arxiv.org/abs/2509.26507>

[3] Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., Lu, M., Song, S., & Abbasi-Yadkori, Y. (2025). Hierarchical Reasoning Model. arXiv:2506.21734.

<https://arxiv.org/abs/2506.21734>

[4] Jolicoeur-Martineau, A. (2025). Less is More: Recursive Reasoning with Tiny Networks. arXiv:2510.04871.

<https://arxiv.org/abs/2510.04871>

[5] Chollet, F., Knoop, M., Kamradt, G., & Landers, B. (2024). ARC Prize 2024: Technical Report. arXiv:2412.04604.

<https://arxiv.org/abs/2412.04604>

[Brief] DataForge 2026: Pathway Track — Problem Statement supplied for this project. It defines the selected concept, submission package, design standards, BDH/BDH-CQ integration guidance, and evidence requirements.

Project note

RuleForge is intentionally narrow. Its value is not the number of pages, rules, or dashboards. The value is that a learner can change the evidence, watch the inferred rule change, test it on something new, and then connect that small experiment to a current research system.